Assignment 2 solution

1. Dynamic array test

- **Test code**

```
module dyn_arr();
    int dyn_arr1 [] = new[5];
    int dyn_arr2 [] = '{9,1,8,3,4,4};

    initial begin
        foreach (dyn_arr1[i]) begin
            dyn_arr1[i] = i;
        end
        $display("dyn_arr1=%p, size=%0d", dyn_arr1, dyn_arr1.size());
        dyn_arr1.delete();
        dyn_arr2.reverse();
        $display("reversed_dyn_arr2=%p", dyn_arr2);
        dyn_arr2.sort();
        $display("sorted_dyn_arr2=%p", dyn_arr2);
        dyn_arr2.rsort();
        $display("reversesort_dyn_arr2=%p", dyn_arr2);
        dyn_arr2.shuffle();
        $display("shuffled_dyn_arr2=%p", dyn_arr2);
        $stop;
    end
endmodule
```

- **Output result**

```
# //
# Loading sv_std.std
# Loading work.dyn_arr(fast)
# dyn_arr1='{0, 1, 2, 3, 4}, size=5
# reversed_dyn_arr2='{4, 4, 3, 8, 1, 9}
# sorted_dyn_arr2='{1, 3, 4, 4, 8, 9}
# reversesort_dyn_arr2='{9, 8, 4, 4, 3, 1}
# shuffled_dyn_arr2='{8, 1, 4, 3, 9, 4}
# ** Note: $stop    : dyn_arr.sv(19)
#    Time: 0 ns  Iteration: 0  Instance: /dyn_arr
# Break in Module dyn_arr at dyn_arr.sv line 19
# Stopped at dyn_arr.sv line 19
VSIM 2>
```

2. Counter

- **Design**

  Design behaves correctly and no bugs detected.

- **Verification plan**

	Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
2	COUNTER_1	assert reset signal and check the count_out bits to be low also check the value of the zero signal to be 1.	direct test in the beginnig of the code	-	a checker in the testbench to make sure the output is correct
3	COUNTER_2	when load is asserted, check the count_out data if it sampled the input data correct or not, also check for max_count to be 1 in case data_load is all ones and zero to be 1 in case data_load is all zeros	direct test by asserting the load_n signal and check the outputs	-	a checker in the testbench to make sure the output is correct
4	COUNTER_3	check the counting procedure in case ce is on and make sure that counter stop counting when the ce is off	direct test by asserting the ce for some iterations then diactivate it for another couple of iterations	-	a checker in the testbench to make sure the output is correct
5	COUNTER_4	check counting procedure for the different values of the up_down signal	direct test by make a full round of counting in both directions	-	a checker in the testbench to make sure the output is correct
6	COUNTER_5	check output values in case of randomizing all the input signals according to the randomization constrains	randomization test by constrainning reset to be off most of the time, load signal to be active 70% of the time, clk enable to be active 70% of the time, unconstraint randomization for input load data and the count direction signal	-	a checker in the testbench to make sure the output is correct

- **Package**

```
package COUNTER_PACKAGE;
    parameter WIDTH = 4;
    class counter_var;
        rand bit rst_n, load_n, up_down, ce;
        rand bit [WIDTH-1:0] data_load;
        constraint c_dist {
            rst_n  dist {1:=95, 0:=5};
            load_n dist {0:=70, 1:=30};
            ce     dist {0:=30, 1:=70};
        }
    endclass
endpackage
```

- **Testbench**

```
import COUNTER_PACKAGE::*;
module counter_tb ();
  //? local parameters
  localparam CLK_CYCLE = 2;
  //? Variables
  int error_count, correct_count;
  //? interface signals
  logic clk;
  logic rst_n;
  logic load_n;
  logic up_down;
  logic ce;
  logic [WIDTH-1:0] data_load;
  logic [WIDTH-1:0] count_out;
  logic max_count;
  logic zero;
  //? debug signals
  logic [WIDTH-1:0] count_out_g;
  logic max_count_g, zero_g;
  //? class object
  counter_var count_rand;
  //? DUT instantiation
  counter #(
      .WIDTH(WIDTH)
  ) COUNTER_DUT (
      .clk(clk),
      .rst_n(rst_n),
      .load_n(load_n),
      .up_down(up_down),
      .ce(ce),
      .data_load(data_load),
      .count_out(count_out),
      .max_count(max_count),
      .zero(zero)
  );
  //? CLK generation
  always #(CLK_CYCLE / 2) clk = ~clk;
  //? Driver block
  initial begin
    //*initialization
    count_rand = new();
    clk = 0;
    rst_n = 1;
    load_n = 1;
    up_down = 0;
    ce = 0;
    data_load = $random;
    #CLK_CYCLE;
    //! COUNTER_1 TEST
    assert_reset();
    $display("COUNTER_1 TEST time = %0t",$time);
```

```
    //! COUNTER_2 TEST
    load_n = 0;
    for(int i=0; i<10; i++) begin
        data_load = $random;
        golden_model();
        check_result(count_out_g, zero_g, max_count_g);
    end
    $display("COUNTER_2 TEST time = %0t",$time);
    //! COUNTER_3 & COUNTER_4 TESTS
    ce = 1;
    up_down = 1;
    for (int i = 0; i<10 ;i++) begin
        golden_model();
        check_result(count_out_g, zero_g, max_count_g);
    end
    up_down = 0;
    for (int i = 0;i<10 ;i++) begin
        golden_model();
        check_result(count_out_g, zero_g, max_count_g);
    end
    ce = 0;
    for (int i = 0;i<5 ;i++ ) begin
        golden_model();
        check_result(count_out_g, zero_g, max_count_g);
    end
    $display("COUNTER_3&4 TEST time = %0t",$time);
    //! COUNTER_5 TEST
    for (int i = 0; i<100; i++) begin
        assert(count_rand.randomize());
        data_load = count_rand.data_load;
        ce = count_rand.ce;
        up_down = count_rand.up_down;
        rst_n = count_rand.rst_n;
        load_n = count_rand.load_n;
        golden_model();
        check_result(count_out_g, zero_g, max_count_g);
    end
    $display("COUNTER_5 TEST time = %0t",$time);
    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end
  //? TASKS
  task assert_reset();
    rst_n = 0;
    count_out_g = 0;
    zero_g = 1;
    max_count_g = 0;
    check_result(count_out_g, zero_g, max_count_g);
    rst_n = 1;
  endtask
```

```
  task check_result(input [WIDTH-1:0] count_out_exp, input zero_exp, max_count_exp);
    #CLK_CYCLE;
    if (count_out_exp == count_out && zero_exp == zero && max_count_exp == max_count) correct_count++;
    else begin
      error_count++;
      $display("xxxx ERROR at time %0t: the actual output result is not matching the expected result xxxx", $time);
      $display("count_out=%0b ,count_out_exp=%0b\nmax_count=%0b, max_count_exp=%0b\nzero=%0b, zero_exp=%0b\ndata_load = %0b, ce = %0b, up_down = %0b \n",
               count_out, count_out_exp, max_count, max_count_exp, zero, zero_exp, data_load, ce, up_down);
    end
  endtask

  task golden_model();
    if(rst_n == 0)begin
        max_count_g = 0;
        zero_g = 1;
        count_out_g = 0;
    end else if(load_n == 0) begin
        count_out_g = data_load;
        if(count_out_g == 0) begin
            max_count_g = 0;
            zero_g = 1;
        end else if(count_out_g == {WIDTH{1'b1}})begin
            max_count_g = 1;
            zero_g = 0;
        end else begin
            max_count_g = 0;
            zero_g = 0;
        end
    end else if(ce == 1) begin
        if(up_down == 1) count_out_g++;
        else count_out_g--;
        if(count_out_g == {WIDTH{1'b1}}) begin
                max_count_g = 1;
                zero_g = 0;
        end else if (count_out_g == 0) begin
            max_count_g = 0;
            zero_g = 1;
        end else begin
            max_count_g = 0;
            zero_g = 0;
        end
    end
  endtask
endmodule
```

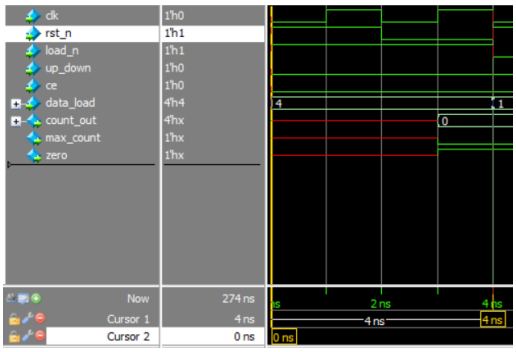
- **DO file**

```
vlib work
vlog counter.v counter_tb.sv COUNTER_PACKAGE.sv +cover -covercells
vsim -voptargs=+acc work.counter_tb -cover
coverage save counter.ucdb -onexit -du counter
#do wave.do
#log -r /*
run -all
quit -sim
vcover report counter.ucdb -details -annotate -all -output cover_rpt.txt
```

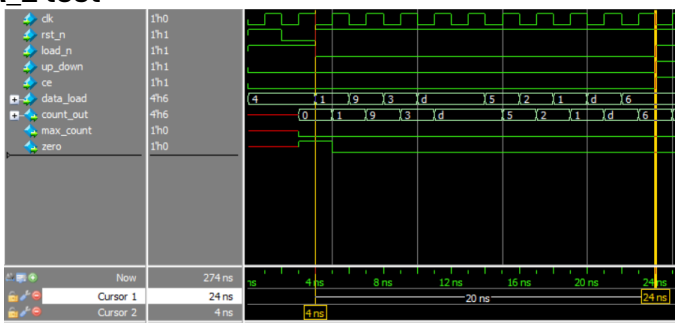
- **Simulation results**

Note: focus on the part of the wave that is located between the cursors

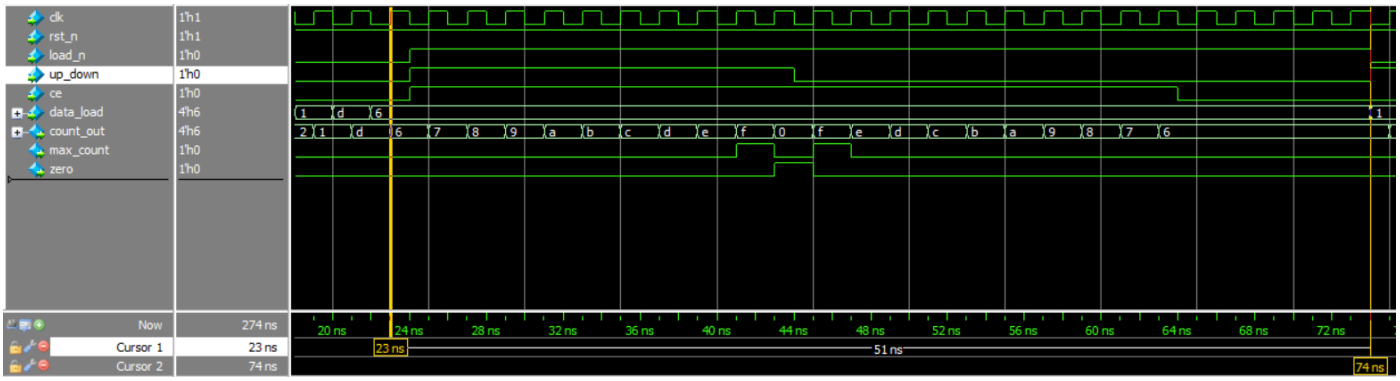
❖ **COUNTER_1 test**



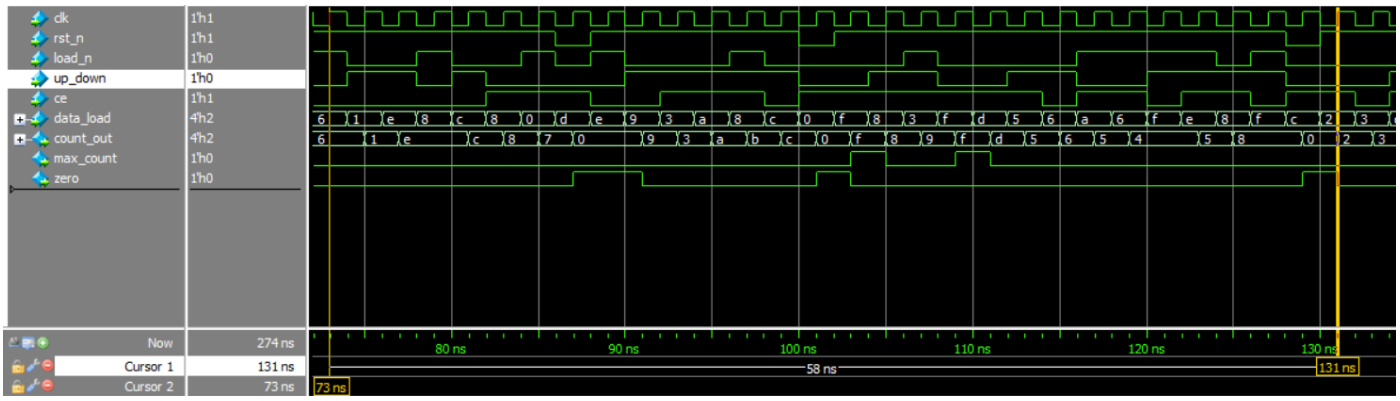
❖ **COUNTER_2 test**



❖ COUNTER_3&4 test



❖ COUNTER_5 test



- ## <u>Code coverage</u>

❖ Branch coverage

```
Coverage Report by instance with details

=================================================================================
=== Instance: /\counter_tb#COUNTER_DUT
=== Design Unit: work.counter
=================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        10        10         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\counter_tb#COUNTER_DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File counter.v
------------------------------------IF Branch------------------------------------
    21                                       115     Count coming in to IF
    21              1                          6         if (!rst_n)

    23              1                         87         else if (!load_n)

    25              1                         12         else if (ce)

                                              10     All False Count
Branch totals: 4 hits of 4 branches = 100.00%

------------------------------------IF Branch------------------------------------
    26                                        12     Count coming in to IF
    26              1                          9             if (up_down)
```

❖ Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\counter_tb#COUNTER_DUT  --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File counter.v
    8                                                module counter (clk ,rst_n, load

    9                                                parameter WIDTH = 4;
```

❖ **Toggle coverage**

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

  ==============================Toggle Details==============================

Toggle Coverage for instance /\counter_tb#COUNTER_DUT  --

                                                  Node      1H->0L      0L->1H  "Coverage"
                                                  ----------------------------------------
                                                    ce           1           1      100.00
                                                   clk           1           1      100.00
                                        count_out[3-0]           1           1      100.00
                                        data_load[0-3]           1           1      100.00
                                                 load_n           1           1      100.00
                                             max_count           1           1      100.00
                                                 rst_n           1           1      100.00
                                               up_down           1           1      100.00
                                                  zero           1           1      100.00

  Total Node Count     =         15
  Toggled Node Count   =         15
  Untoggled Node Count =          0

  Toggle Coverage      =     100.00% (30 of 30 bins)


  Total Coverage By Instance (filtered view): 100.00%
```

3. ALSU

- **Design**

BUGS: → full addition is not working properly as Cin is not added with A and B when an ADD operation is raised.

→ When any of the bypass ports are raised while an invalid operation occurring, the output leds start blinking rather than simply bypassing the required value to output and keeping leds low. Bypass operation is higher priority so this case must be treated as a valid scenario.

→ Input opcode register is missing.

```
//leds output blinking
always @(posedge clk or posedge rst) begin
  if(rst) begin
      leds <= 0;
  end else begin
      if (invalid)
        leds <= ~leds;
      else
        leds <= 0;
  end
end
```

```
//leds output blinking
always @(posedge clk or posedge rst) begin
  if(rst) begin
      leds <= 0;
  end else begin
      if (invalid && !bypass_A_reg && !bypass_B_reg)
        leds <= ~leds;
      else
        leds <= 0;
  end
end
```

```
case (opcode)
  3'h0: begin
```

```
else begin
    case (opcode_reg)
      3'h0: begin
```

```
3'h2: out <= A_reg + B_reg;
3'h3: out <= A_reg * B_reg;
```

```
3'h2: out <= A_reg + B_reg + cin_reg;
3'h3: out <= A_reg * B_reg;
```

- ## Verification plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	When the reset is asserted, the output should be low	Direct at the start of the sim, then randomize with constraint that drive the reset to be off most of the simulation time	-	a checker in the testbench to make sure the output is correct
ALSU_2	In case of invalid cases don't occur, when opcode is add, then out should perform the addition on ports A and B taking cin into consediration as FULL_ADDER parameter is on	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time	-	output checked against golden model
ALSU_3	In case of invalid cases don't occur, when opcode is mult, then out should perform the multiplication on ports A and B	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time	-	output checked against golden model
ALSU_4	In case of invalid cases don't occur, when opcode is OR or XOR, then out should perform the bitwise oring or xoring operation on ports A and B	Randomization without constraints on A and B	-	output checked against golden model
ALSU_5	In case of invalid cases don't occur, when opcode is SHIFT, then out should perform shift operation on the current out value in the direction specified in the input direction value with adding the serial_in value in the shifted location	Randomization without constraints on serial_in and direction values	-	output checked against golden model
ALSU_6	In case of invalid cases don't occur, when opcode is ROTATE, then out should perform rotate operation to the out in the direction specified by the input direction signal	Randomization without constraints on direction values	-	output checked against golden model
ALSU_7	In case of invalid cases don't occur, when opcode is OR or XOR and any reduction signal is asserted, out should perform reduction OR or XOR on the specified port	Randomization under constraints on the selected port to perform reduction operation to have only one bit high in its 3bits most of the time and constraint the other port bits to be low	-	output checked against golden model
ALSU_8	when any of the bypassing port is asserted, out should take the value of the specified port to be bypassed	Randomization without constraints on A,B, red_op and opcode	-	output checked against golden model
ALSU_9	try invalid cases, wrong opcode and true opcode except OR or XOR with any of the reduction ports is asserted	Direct test by trying to enter the stimilus of the invalid cases manually	-	output checked against golden model
ALSU_10	Try to randomize all the inputs under certain number of constraints for a while to detect any strange behaviour not covered in the previous test cases	* Randomization under constrains on A,B to be MAXPOS, MAXNEG, ZERO most of the time in case of ADD or MULT operations and one bit only is high in one of them in case of reduction is used. * Randomization under constraints on opcode to make 110 and 111 not occured most of the time. * Randomization under constraints on bypass and reset to be disabled most of the time. * Randomization under constraints on reduction ports to be disapled most if the time except if the current operation is oring or xoring. * Randomization without constraints on Cin, serial_in and direction ports.	-	output checked against golden model

- ## Package

```
package ALSU_PACKAGE;
    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    parameter MAXPOS = 3'b011;
    parameter MAXNEG = 3'b100;
    parameter ZERO = 3'b000;
    typedef enum logic [2:0] { OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7 } opcode_e;
    class ALSU_Rand_Var;
        rand bit rst, cin, serial_in, direction, red_op_A, red_op_B, bypass_A, bypass_B;
        rand opcode_e opcode;
        rand bit signed [2:0] A,B;

        function new();
            rst = 0;
            cin = 0;
            serial_in = 0;
            direction = 0;
            red_op_A = 0;
            red_op_B = 0;
            bypass_A = 0;
            bypass_B = 0;
            opcode = opcode_e'($random);
            A = $random;
            B = $random;
        endfunction

        constraint constraints {
            rst dist {0:=99, 1:=1};
            bypass_A dist {0:=99, 1:=1};
            bypass_B dist {0:=99, 1:=1};
            opcode dist {INVALID_6:=10, INVALID_7:=10, [OR:ROTATE]:=90};
            if(opcode == OR || opcode == XOR){
                red_op_A dist {1:=20, 0:=80};
                red_op_B dist {1:=20, 0:=80};
                if(red_op_A) {
                    $onehot(A);
                    B == 0;
                } else if(red_op_B){
                    $onehot(B);
                    A == 0;
                }
            } else if(opcode == ADD || opcode == MULT) {
                A dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                B dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                red_op_A dist {1:=1, 0:=99};
                red_op_B dist {1:=1, 0:=99};
            } else {
                red_op_A dist {1:=1, 0:=99};
                red_op_B dist {1:=1, 0:=99};
            }
        }
    endclass
endpackage
```

- **Testbench**

```
import ALSU_PACKAGE::*;
module ALSU_tb ();
  //? local parameters
  localparam CLK_CYCLE = 2;
  //? Variables
  int error_count, correct_count;
  //? interface signals
  logic clk;
  logic rst;
  logic cin;
  logic serial_in;
  logic direction;
  logic red_op_A;
  logic red_op_B;
  logic bypass_A;
  logic bypass_B;
  logic signed [2:0] A;
  logic signed [2:0] B;
  logic [5:0] out;
  logic [15:0] leds;
  opcode_e opcode;
  //? golden model outputs
  logic signed [5:0] out_g;
  logic [15:0] leds_g;
  //? input registers
  logic cin_reg;
  logic serial_in_reg;
  logic direction_reg;
  logic red_op_A_reg;
  logic red_op_B_reg;
  logic bypass_A_reg;
  logic bypass_B_reg;
  logic signed [2:0] A_reg;
  logic signed [2:0] B_reg;
  opcode_e opcode_reg;
  //? class object
  ALSU_Rand_Var ALSU_obj;
  //? DUT instantiation
  ALSU #(
      .INPUT_PRIORITY(INPUT_PRIORITY),
      .FULL_ADDER(FULL_ADDER)
  ) ALSU_DUT (
      .clk(clk),
      .rst(rst),
      .cin(cin),
      .serial_in(serial_in),
      .direction(direction),
      .red_op_A(red_op_A),
      .red_op_B(red_op_B),
      .bypass_A(bypass_A),
      .bypass_B(bypass_B),
      .A(A),
```

```
        .A(A),
        .B(B),
        .opcode(opcode),
        .out(out),
        .leds(leds)
    );
    //? CLK generation
    always #(CLK_CYCLE / 2) clk = ~clk;
    //? Driver block
    initial begin
      //*initialization
      ALSU_obj = new();
      clk = 0;
      assign_class_properties();
      //! ALSU_1 TEST
      assert_reset();
      $display("ALSU_1 TEST time = %0t", $time);
      //! ALSU_2 TEST
      ALSU_obj.opcode = ADD;
      for (int i = 0; i < 20; i++) begin
        assert (ALSU_obj.randomize(A, B, cin));
        run_test();
      end
      $display("ALSU_2 TEST time = %0t", $time);
      //! ALSU_3 TEST
      ALSU_obj.opcode = MULT;
      for (int i = 0; i < 20; i++) begin
        assert (ALSU_obj.randomize(A, B));
        run_test();
      end
      $display("ALSU_3 TEST time = %0t", $time);
      //! ALSU_4 TEST
      for (int i = 0; i < 20; i++) begin
        ALSU_obj.opcode = OR;
        assert (ALSU_obj.randomize(A, B));
        run_test();
        ALSU_obj.opcode = XOR;
        run_test();
      end
      $display("ALSU_4 TEST time = %0t", $time);
      //! ALSU_5 TEST
      ALSU_obj.opcode = SHIFT;
      for (int i = 0; i < 20; i++) begin
        assert (ALSU_obj.randomize(serial_in, direction));
        run_test();
      end
      $display("ALSU_5 TEST time = %0t", $time);
```

```
      //! ALSU_6 TEST
      ALSU_obj.opcode = ROTATE;
      for (int i = 0; i < 20; i++) begin
        assert (ALSU_obj.randomize(direction));
        run_test();
      end
      $display("ALSU_6 TEST time = %0t", $time);
```

```
//! ALSU_7 TEST
for (int j = 0; j < 2; j++) begin
  if (j == 0) ALSU_obj.opcode = OR;
  else ALSU_obj.opcode = XOR;
  for (int i = 0; i < 30; i++) begin
    if (i < 10) begin
      ALSU_obj.red_op_A = 1;
      ALSU_obj.red_op_B = 0;
    end else if (i < 20) begin
      ALSU_obj.red_op_A = 0;
      ALSU_obj.red_op_B = 1;
    end else begin
      ALSU_obj.red_op_A = 1;
      ALSU_obj.red_op_B = 1;
    end
    assert (ALSU_obj.randomize(A, B));
    run_test();
  end
end
$display("ALSU_7 TEST time = %0t", $time);
//! ALSU_8 TEST
for (int i = 0; i < 30; i++) begin
  if (i < 10) begin
    ALSU_obj.bypass_A = 1;
    ALSU_obj.bypass_B = 0;
  end else if (i < 20) begin
    ALSU_obj.bypass_A = 0;
    ALSU_obj.bypass_B = 1;
  end else begin
    ALSU_obj.bypass_A = 1;
    ALSU_obj.bypass_B = 1;
  end
  assert (ALSU_obj.randomize(A, B, red_op_A, red_op_B, opcode));
  run_test();
end
$display("ALSU_8 TEST time = %0t", $time);
//! ALSU_9 TEST
ALSU_obj.bypass_A = 0;
ALSU_obj.bypass_B = 0;
ALSU_obj.opcode   = INVALID_6;
ALSU_obj.red_op_A = 1;
run_test();
ALSU_obj.opcode = ADD;
run_test();
for(int i =0; i<5;i++)begin
    ALSU_obj.opcode   = INVALID_7;
    ALSU_obj.red_op_A = 0;
    ALSU_obj.red_op_B = 0;
    run_test();
end
$display("ALSU_9 TEST time = %0t", $time);
```

```
    //! ALU_10 TEST
    for (int i = 0; i < 100; i++) begin
      assert (ALSU_obj.randomize());
      run_test();
    end
    $display("ALSU_10 TEST time = %0t", $time);
    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end
  //? TASKS
  task assert_reset();
    rst = 1;
    #CLK_CYCLE;
    check_result(out_g, leds_g);
    rst = 0;
endtask

task automatic check_result(input [5:0] out_exp, input [15:0] leds_exp);
if (out_exp == out && leds_exp == leds) correct_count++;
else begin
    error_count++;
      $display(
          "xxxx ERROR at time %0t: the actual output result is not matching the expected result xxxx",
            $time);
            $display("out=%0b ,out_exp=%0b, leds=%0h, leds_exp=%0h", out, out_exp, leds, leds_exp);
        end
    endtask

    task assign_class_properties();
      rst = ALSU_obj.rst;
      A = ALSU_obj.A;
      B = ALSU_obj.B;
      cin = ALSU_obj.cin;
      opcode = ALSU_obj.opcode;
      serial_in = ALSU_obj.serial_in;
      direction = ALSU_obj.direction;
      red_op_A = ALSU_obj.red_op_A;
      red_op_B = ALSU_obj.red_op_B;
      bypass_A = ALSU_obj.bypass_A;
      bypass_B = ALSU_obj.bypass_B;
    endtask

    task run_test();
      assign_class_properties();
      #(2 * CLK_CYCLE);
      check_result(out_g, leds_g);
    endtask
```

```
//*GOLDEN MODEL
always @(posedge clk or posedge rst) begin
  if (rst) begin
    cin_reg <= 0;
    red_op_B_reg <= 0;
    red_op_A_reg <= 0;
    bypass_B_reg <= 0;
    bypass_A_reg <= 0;
    direction_reg <= 0;
    serial_in_reg <= 0;
    opcode_reg <= ADD;
    A_reg <= 0;
    B_reg <= 0;
  end else begin
    cin_reg <= cin;
    red_op_B_reg <= red_op_B;
    red_op_A_reg <= red_op_A;
    bypass_B_reg <= bypass_B;
    bypass_A_reg <= bypass_A;
    direction_reg <= direction;
    serial_in_reg <= serial_in;
    opcode_reg <= opcode;
    A_reg <= A;
    B_reg <= B;
  end
end
always @(posedge clk or posedge rst) begin
  if (rst) begin
    out_g  <= 0;
    leds_g <= 0;
  end else if (bypass_A_reg || bypass_B_reg) begin
    if (INPUT_PRIORITY == "A") begin
      if (bypass_A_reg) begin
        out_g  <= A_reg;
        leds_g <= 0;
      end else begin
        out_g  <= B_reg;
        leds_g <= 0;
      end
    end else begin
      if (bypass_B_reg) begin
        out_g  <= B_reg;
        leds_g <= 0;
      end else begin
        out_g  <= A_reg;
        leds_g <= 0;
      end
    end
  end else begin
    case (opcode_reg)
```

```
        case (opcode_reg)
          ADD: begin
            if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
              if (FULL_ADDER == "ON") begin
                out_g  <= {{3{A_reg[2]}}, A_reg} + {{3{B_reg[2]}}, B_reg} + cin_reg;
                leds_g <= 0;
              end else begin
                out_g  <= {{3{A_reg[2]}}, A_reg} + {{3{B_reg[2]}}, B_reg};
                leds_g <= 0;
              end
            end else begin
              out_g  <= 0;
              leds_g <= ~leds_g;
            end
          end
          MULT: begin
            if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
              out_g  <= A_reg * B_reg;
              leds_g <= 0;
            end else begin
              out_g  <= 0;
              leds_g <= ~leds_g;
            end
          end
          SHIFT: begin
            if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
              if (direction_reg) out_g <= {out_g[4:0], serial_in_reg};
              else out_g <= {serial_in_reg, out_g[5:1]};
              leds_g <= 0;
            end else begin
              out_g  <= 0;
              leds_g <= ~leds_g;
            end
          end
          ROTATE: begin
            if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
              if (direction_reg) out_g <= {out_g[4:0], out_g[5]};
              else out_g <= {out_g[0], out_g[5:1]};
              leds_g <= 0;
            end else begin
              out_g  <= 0;
              leds_g <= ~leds_g;
            end
          end
          OR: begin
```

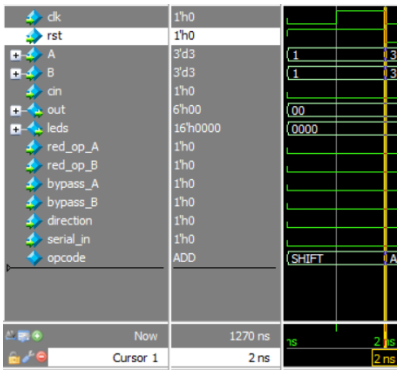
```
            OR: begin
              if (red_op_A_reg || red_op_B_reg) begin
                if (INPUT_PRIORITY == "A") begin
                  if (red_op_A_reg) out_g <= |A_reg;
                  else out_g <= |B_reg;
                end else begin
                  if (red_op_B_reg) out_g <= |B_reg;
                  else out_g <= |A_reg;
                end
              end else out_g <= A_reg | B_reg;
              leds_g <= 0;
            end
            XOR: begin
              if (red_op_A_reg || red_op_B_reg) begin
                if (INPUT_PRIORITY == "A") begin
                  if (red_op_A_reg) out_g <= ^A_reg;
                  else out_g <= ^B_reg;
                end else begin
                  if (red_op_B_reg) out_g <= ^B_reg;
                  else out_g <= ^A_reg;
                end
              end else out_g <= A_reg ^ B_reg;
              leds_g <= 0;
            end
            default: begin
              out_g  <= 0;
              leds_g <= ~leds_g;
            end
          endcase
        end
      end


    endmodule
```

- **Do file**

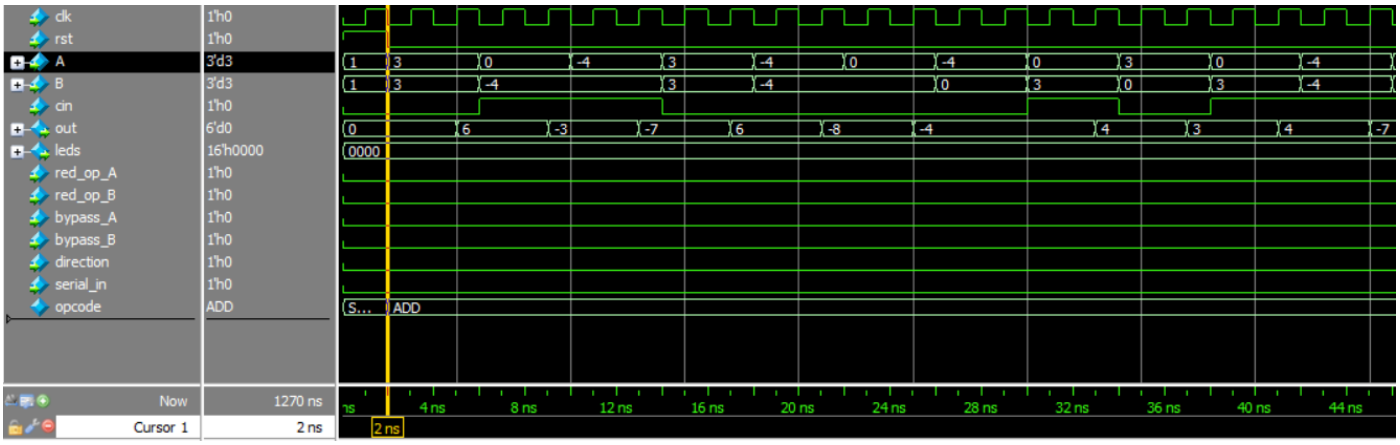
```
vlib work
vlog ALSU_PACKAGE.sv ALSU.v ALSU_tb.sv +cover -covercells
vsim -voptargs=+acc work.ALSU_tb -cover
coverage save ALSU.ucdb -onexit -du ALSU
do wave.do
log -r /*
run -all
coverage exclude -src ALSU.v -line 76 -code b
coverage exclude -du ALSU -togglenode {cin_reg[1]}
quit -sim
vcover report ALSU.ucdb -details -annotate -all -output cover_rpt.txt
```

- **Simulation results**

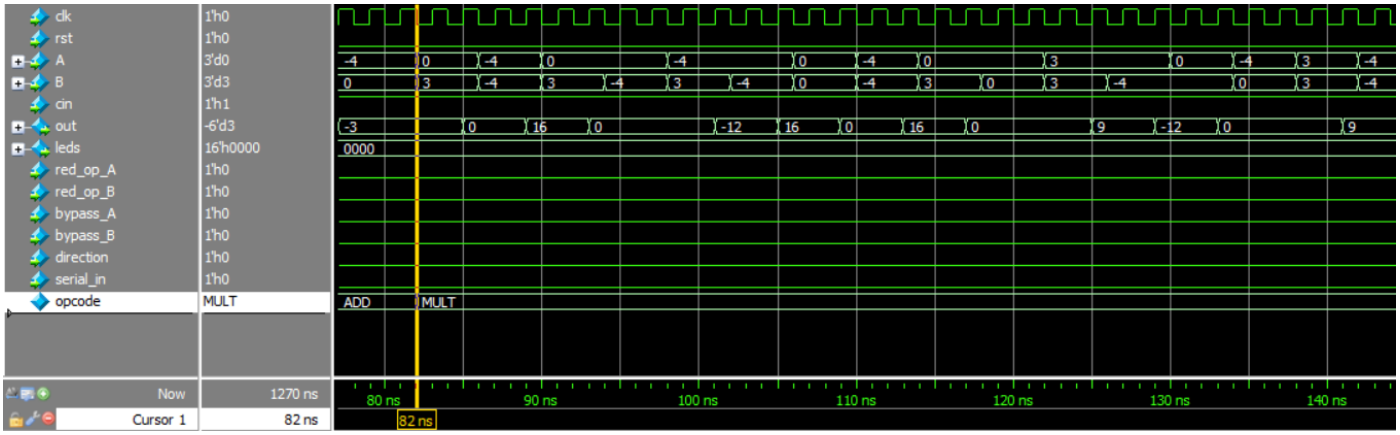
❖ **ALSU_1**



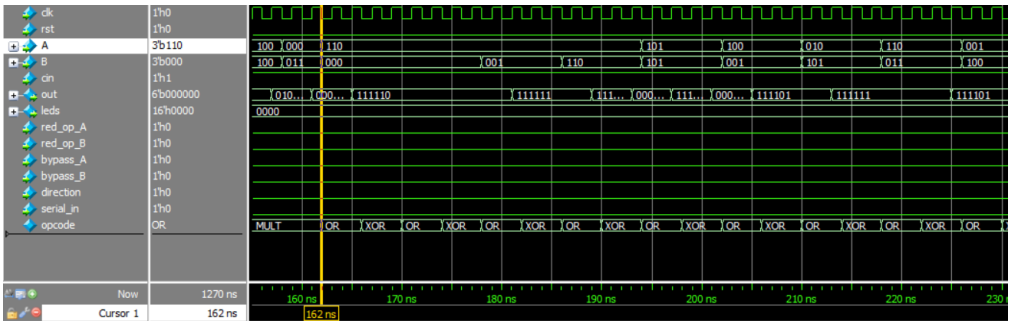
❖ **ALSU_2**



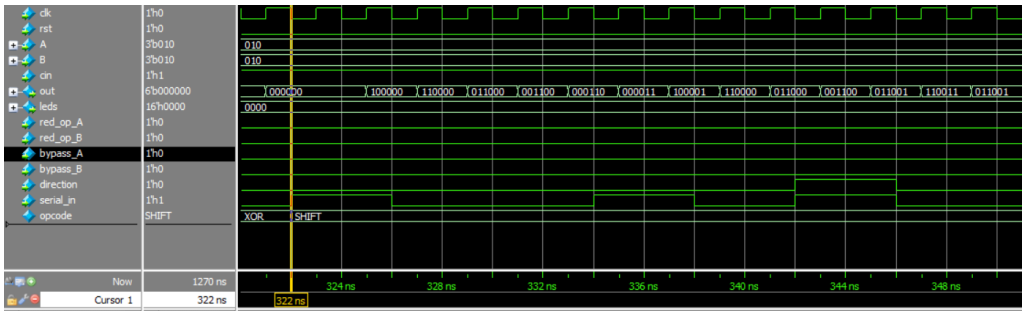
❖ **ALSU_3**



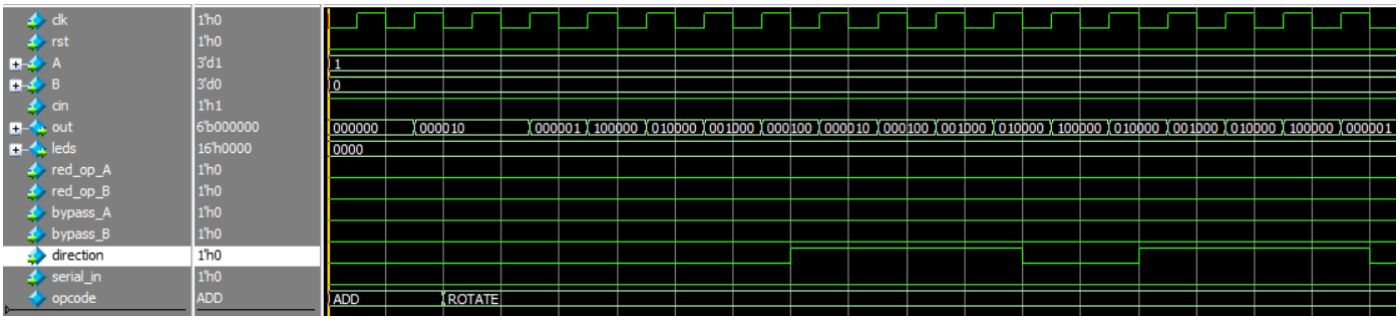
❖ ALSU_4



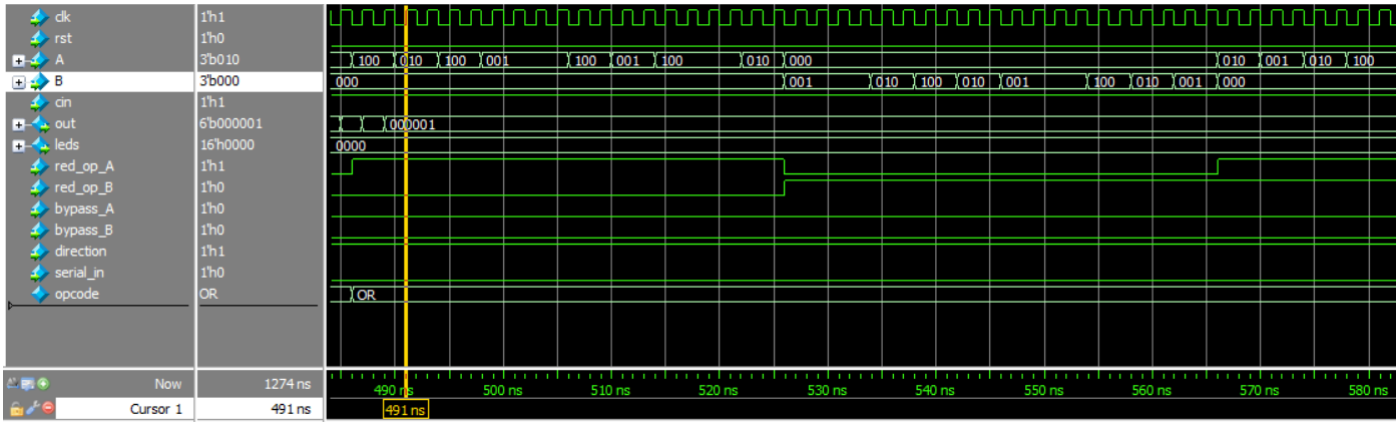
❖ ALSU_5



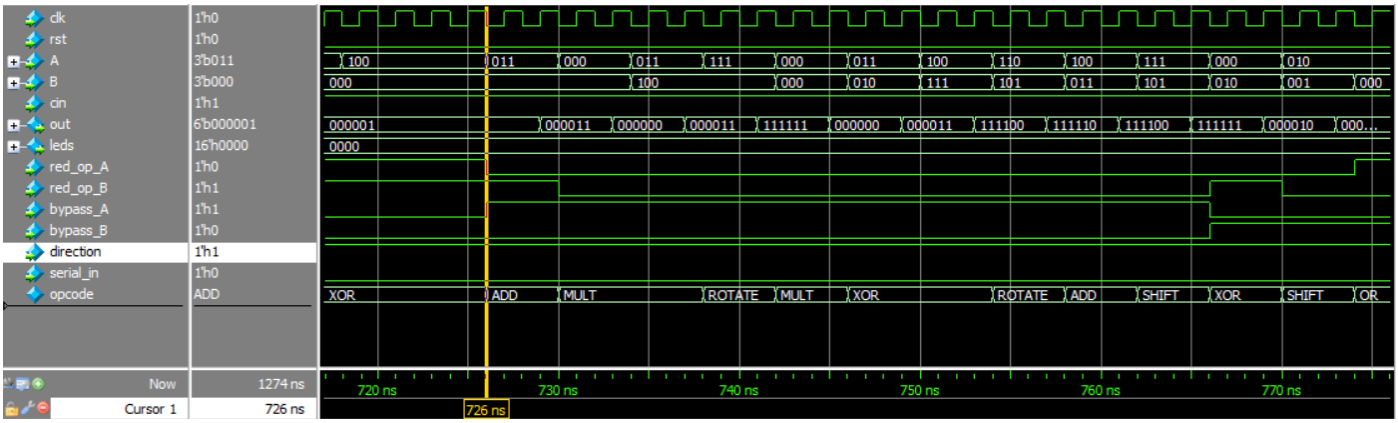
❖ ALSU_6



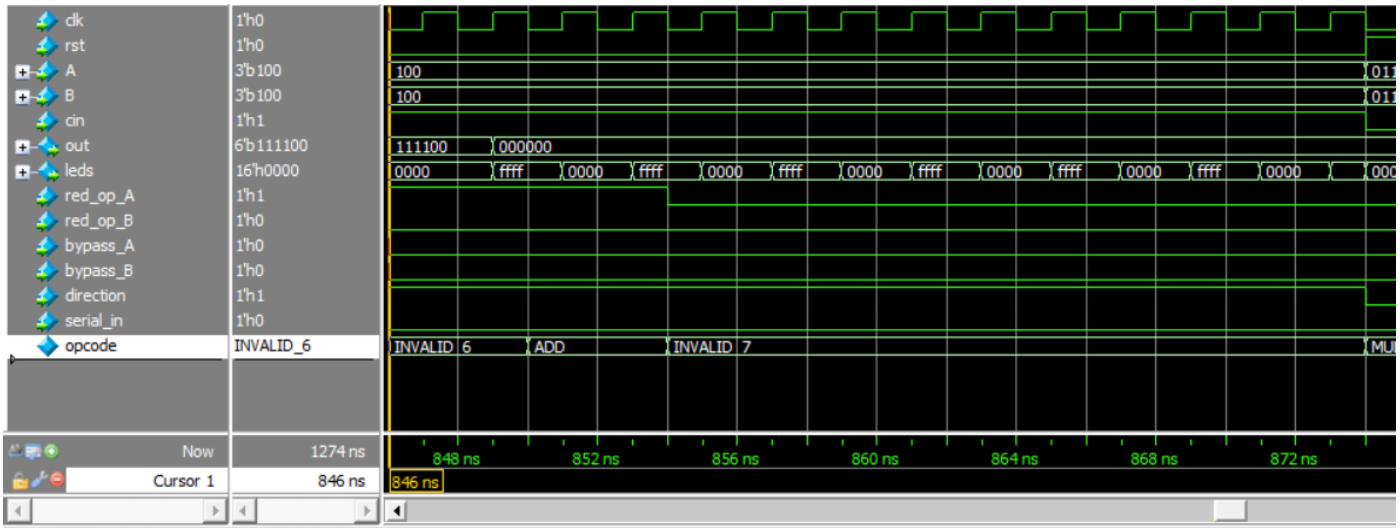
❖ ALSU_7



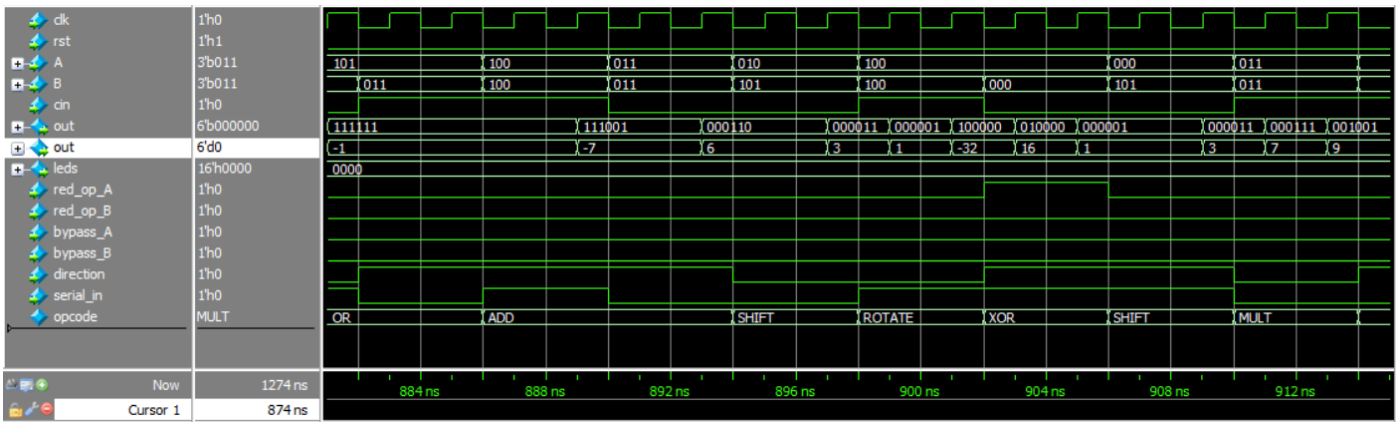
❖ **ALSU_8**



❖ **ALSU_9**



❖ **ALSU_10**



- **<u>Code coverage</u>**

  ➔ We added two exclusions, exclusion on the toggle node cin_reg[1] as it is used as a positive sign bit that normally to be always zero, this bit is added to make the signed full addition operation perform well without bugs like unsigned promotion for example that happens when adding mix of signed and unsigned numbers.

➔ The case statement does not include a default branch to handle invalid opcode values, which prevented the tool from reaching 100% coverage. Specifically, the scenario where none of the case branches are true is not explicitly covered. However, this situation is already handled by the surrounding if-else hierarchy, since the code only enters the case statement when the opcode is valid. Therefore, adding a default branch is unnecessary, as it would never be executed regardless of whether it is included or not. Look at the following snippet and you will see what I am saying.

```
//Invalid handling
assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op | invalid_opcode;
```

```
else if (invalid)
    out <= 0;
else begin
    case (opcode_reg)
```

❖ **Branch coverage**

```
Coverage Report by instance with details

================================================================================
=== Instance: /\ALSU_tb#ALSU_DUT
=== Design Unit: work.ALSU
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        25        25         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\ALSU_tb#ALSU_DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ALSU.v
------------------------------------IF Branch------------------------------------
    24                                       558     Count coming in to IF
    24             1                           4       if(rst) begin

    35             1                         554       end else begin

Branch totals: 2 hits of 2 branches = 100.00%
```

❖ **Statement coverage**

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\ALSU_tb#ALSU_DUT  --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ALSU.v
    1                                                module ALSU(A, B, cin, serial_in, red_op

    2                                                parameter INPUT_PRIORITY = "A";
```

❖ **Toggle coverage**

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        118       118         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\ALSU_tb#ALSU_DUT  --

                                  Node      1H->0L      0L->1H                "Coverage"
                                  ----------------------------------------------------------
                                 A[0-2]          1           1                    100.00
                             A_reg[2-0]          1           1                    100.00
                                 B[0-2]          1           1                    100.00
                             B_reg[2-0]          1           1                    100.00
                               bypass_A          1           1                    100.00
                           bypass_A_reg          1           1                    100.00
                               bypass_B          1           1                    100.00
                           bypass_B_reg          1           1                    100.00
                                    cin          1           1                    100.00
                            cin_reg[0]          1           1                    100.00
                                    clk          1           1                    100.00
                              direction          1           1                    100.00
                          direction_reg          1           1                    100.00
                                invalid          1           1                    100.00
                           invalid_opcode          1           1                    100.00
                           invalid_red_op          1           1                    100.00
                               leds[15-0]          1           1                    100.00
                             opcode[0-2]          1           1                    100.00
                          opcode_reg[2-0]          1           1                    100.00
                                 out[5-0]          1           1                    100.00
                                 red_op_A          1           1                    100.00
                             red_op_A_reg          1           1                    100.00
                                 red_op_B          1           1                    100.00
                             red_op_B_reg          1           1                    100.00
                                      rst          1           1                    100.00
                                 serial_in          1           1                    100.00
                             serial_in_reg          1           1                    100.00

Total Node Count     =         59
Toggled Node Count   =         59
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (118 of 118 bins)
```